



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

Práctica 7: Chatbot

NATURAL LANGUAGE PROCESSING

7CM2

INTEGRANTES:

VÁZQUEZ BLANCAS CESAR SAID

PROFESOR:

JORGE LUIS ROSAS TRIGUEROS

INSTITUTO POLITÉCNICO NACIONAL



FECHA DE ENTREGA: 25 DE MAYO DE 2026

FECHA DE REALIZACION DE LA PRACTICA: 17 DE MAYO DE 2026

Marco Teórico

El Procesamiento de Lenguaje Natural, conocido como NLP por sus siglas en inglés, es una rama de la inteligencia artificial enfocada en el desarrollo de modelos y técnicas que permiten a las computadoras analizar, interpretar, generar y responder lenguaje humano de manera automática. Esta área combina conocimientos de lingüística, aprendizaje automático y ciencias de la computación, permitiendo crear sistemas capaces de trabajar con texto escrito por usuarios en contextos reales.

Dentro del NLP, una de las aplicaciones más importantes es el desarrollo de chatbots, los cuales son sistemas conversacionales capaces de recibir mensajes de un usuario y generar respuestas adecuadas de acuerdo con una tarea específica. Estos sistemas pueden utilizarse en atención al cliente, educación, soporte técnico, asistentes virtuales y automatización de procesos. En el contexto de esta práctica, el chatbot desarrollado está orientado al área de soporte técnico, también conocida como IT Helpdesk.

Un chatbot de IT Helpdesk tiene como objetivo recibir reportes o tickets de soporte técnico escritos en lenguaje natural, identificar el posible problema y proporcionar una solución inicial. Este tipo de sistema puede ayudar a resolver incidencias comunes relacionadas con conexión a internet, contraseñas, impresoras, correo electrónico, software, dispositivos periféricos, errores de sistema y problemas de seguridad. De esta forma, el chatbot funciona como un primer nivel de atención, capaz de orientar al usuario antes de escalar el caso a personal especializado.

El conjunto de datos utilizado en esta práctica está basado en tickets de soporte técnico. Cada ejemplo contiene una entrada que representa el problema reportado por el usuario y una salida esperada que incluye un diagnóstico y una solución. Este formato permite que el modelo aprenda a relacionar distintos tipos de problemas técnicos con respuestas estructuradas, manteniendo una forma de salida clara mediante el patrón: *Diagnosis* y *Solution*.

Tradicionalmente, los sistemas de soporte automatizado podían construirse mediante reglas fijas o árboles de decisión. Sin embargo, estos enfoques suelen ser limitados porque dependen de frases exactas o condiciones definidas manualmente. Si el usuario expresa el problema de una forma diferente, el sistema puede fallar o no reconocer la intención. Por esta razón, los modelos modernos de lenguaje representan una alternativa más flexible, ya que pueden interpretar variaciones lingüísticas y generar respuestas más naturales.

Los modelos de lenguaje actuales se basan principalmente en la arquitectura Transformer, la cual introdujo el mecanismo de atención o *self-attention*. Este mecanismo permite que el modelo analice la relación entre las palabras de una secuencia y comprenda mejor el contexto completo de una entrada. Gracias a esta arquitectura, los modelos Transformer

han demostrado un alto desempeño en tareas como traducción automática, resumen de texto, clasificación, generación de respuestas y sistemas conversacionales.

En esta práctica se utilizó un modelo preentrenado de la familia LLaMA, específicamente unsloth/Llama-3.2-1B-Instruct. Este tipo de modelo pertenece a la categoría de modelos de lenguaje autoregresivos, ya que genera texto token por token a partir del contexto previo. Al tratarse de una versión *Instruct*, el modelo ya cuenta con una capacidad previa para seguir instrucciones y responder de manera conversacional, por lo que resulta adecuado para construir un chatbot especializado.

Para adaptar el modelo al dominio específico de soporte técnico, se aplicó un proceso de ajuste fino o *fine-tuning*. El fine-tuning consiste en tomar un modelo previamente entrenado con grandes cantidades de texto y ajustarlo con un conjunto de datos más pequeño y especializado. En este caso, el objetivo del ajuste fue enseñar al modelo a responder como un chatbot de IT Helpdesk, generando diagnósticos breves y soluciones claras para problemas técnicos.

Debido a que entrenar completamente un modelo de lenguaje puede requerir una gran cantidad de memoria y recursos computacionales, se utilizó la técnica LoRA, o *Low-Rank Adaptation*. LoRA permite realizar un ajuste eficiente del modelo modificando únicamente una pequeña cantidad de parámetros adicionales, sin alterar todos los pesos originales del modelo base. Esto reduce el costo computacional del entrenamiento y permite adaptar modelos grandes con recursos más limitados.

En la implementación realizada, LoRA se aplicó sobre módulos internos del modelo relacionados con la atención, específicamente q_proj, k_proj, v_proj y o_proj. Estos componentes participan en el procesamiento de las relaciones contextuales dentro del Transformer. Al ajustar estos módulos, el modelo puede adaptar su comportamiento a la tarea de soporte técnico sin necesidad de reentrenar todos sus parámetros. Esta configuración se encuentra en el código de entrenamiento, donde se define el uso de LoRA con valores como $r=8$, $\text{lo_alpha}=16$ y $\text{lo_dropout}=0.05$.

El entrenamiento fue estructurado en formato conversacional. Para cada ejemplo del dataset se construyó un mensaje de sistema que indica al modelo su rol como chatbot de soporte técnico, un mensaje de usuario con el problema reportado y una respuesta del asistente con el diagnóstico y la solución. Este formato permite que el modelo aprenda no solo el contenido de las respuestas, sino también el comportamiento esperado dentro de una conversación. En el código, este comportamiento se define mediante un mensaje de sistema que indica que el chatbot debe responder en inglés y utilizar el formato Diagnosis: ... Solution:

Durante la preparación de los datos, se aplicó tokenización para convertir el texto en una representación numérica que el modelo pudiera procesar. Además, se utilizó una estrategia de enmascaramiento para que el modelo no aprendiera a repetir el mensaje del

sistema ni la entrada del usuario, sino únicamente la respuesta esperada del asistente. Esta técnica ayuda a que el modelo se enfoque en generar la salida correspondiente al diagnóstico y la solución.

Finalmente, el modelo ajustado fue guardado en Google Drive y posteriormente fusionado con el modelo base. Esto permitió obtener una versión lista para ser cargada y probada sin tener que repetir el entrenamiento. Para la evaluación, se desarrolló una interfaz con Gradio, en la cual el usuario puede escribir problemas técnicos y recibir respuestas generadas por el chatbot. Además, la interfaz incluye pruebas automáticas con ejemplos representativos, como problemas de WiFi, lentitud del equipo, contraseñas, impresoras, correo electrónico, cámara, malware y errores de plataforma.

En conjunto, esta práctica demuestra cómo un modelo de lenguaje preentrenado puede ser adaptado mediante LoRA para desempeñar una tarea específica de soporte técnico. El sistema resultante permite diagnosticar problemas comunes y sugerir soluciones iniciales, simulando el comportamiento de un chatbot de primer nivel dentro de un área de IT Helpdesk.

.

Material y Equipo

Para la realización de esta práctica se emplearon diversas herramientas de software orientadas al entrenamiento, adaptación y prueba de modelos de lenguaje aplicados al procesamiento de lenguaje natural. El objetivo principal fue construir un chatbot especializado en soporte técnico, capaz de diagnosticar problemas reportados por usuarios y sugerir soluciones iniciales.

En primer lugar, se utilizó el lenguaje de programación Python, debido a su amplia adopción en el desarrollo de aplicaciones de inteligencia artificial, aprendizaje automático y procesamiento de lenguaje natural. Python permitió integrar de manera sencilla las bibliotecas necesarias para cargar el dataset, preparar los datos, entrenar el modelo y construir una interfaz de prueba.

El desarrollo y entrenamiento del modelo se llevó a cabo en Google Colab, ya que esta plataforma permite ejecutar código Python en la nube y utilizar recursos de GPU para acelerar el proceso de entrenamiento. Además, se empleó Google Drive como medio de almacenamiento para guardar el dataset, el adaptador LoRA entrenado y el modelo fusionado, evitando la pérdida de archivos al finalizar la sesión de Colab.

Para el manejo del modelo de lenguaje se utilizó la biblioteca Transformers de Hugging Face. Esta biblioteca permite cargar modelos preentrenados, tokenizadores y clases especializadas para tareas de generación de texto. En esta práctica se emplearon las clases AutoTokenizer y AutoModelForCausalLM, las cuales permitieron cargar el tokenizador y el modelo base utilizado para el chatbot. En el código de entrenamiento se observa que el modelo base seleccionado fue unsloth/Llama-3.2-1B-Instruct, el cual fue cargado mediante la biblioteca Transformers.

También se utilizó la biblioteca Datasets de Hugging Face, la cual facilitó la carga del conjunto de datos almacenado en formato JSON. Esta herramienta permitió convertir el archivo del dataset en una estructura compatible con el proceso de entrenamiento. En el código, el dataset se carga desde Google Drive mediante load_dataset, usando la ruta /content/drive/MyDrive/datasets/it_helpdesk_dataset_finetuning.json.

Para realizar el ajuste fino eficiente del modelo se utilizó la biblioteca PEFT (*Parameter Efficient Fine-Tuning*). Esta biblioteca permite aplicar técnicas como LoRA (*Low-Rank Adaptation*), con la finalidad de adaptar un modelo preentrenado a una tarea específica sin modificar todos sus parámetros. En esta práctica, LoRA fue utilizado para especializar el modelo en la tarea de soporte técnico, reduciendo el costo computacional del entrenamiento. En el código se importan LoraConfig, get_peft_model y PeftModel, herramientas necesarias para configurar, aplicar y posteriormente cargar el adaptador LoRA.

El entrenamiento se llevó a cabo utilizando PyTorch, framework que permite trabajar con tensores, modelos neuronales y operaciones aceleradas por GPU. PyTorch fue utilizado como base para la carga del modelo, la configuración de precisión float16, el entrenamiento y la liberación de memoria GPU al fusionar el modelo. En el código también se emplea `torch.cuda.empty_cache()` para limpiar memoria antes de cargar nuevamente el modelo base durante la fusión del adaptador LoRA.

Además, se utilizaron bibliotecas auxiliares como JSON, para la lectura del dataset; OS, para la validación de rutas y directorios; y Shutil, para eliminar carpetas previas de entrenamiento y evitar conflictos con modelos anteriores. Estas herramientas permitieron preparar correctamente el entorno antes de iniciar el entrenamiento.

Para la prueba del modelo se utilizó Gradio, una biblioteca que permite crear interfaces gráficas de manera sencilla para modelos de machine learning. Con Gradio se desarrolló una interfaz web en la que el usuario puede escribir un problema técnico y recibir una respuesta generada por el chatbot. La interfaz también incluye botones de ejemplo y pruebas automáticas para validar el comportamiento del modelo. En el código de prueba se observa que la interfaz fue construida mediante `gr.Blocks`, `gr.Chatbot`, `gr.Textbox` y botones interactivos.

Modelo Utilizado

El modelo base utilizado en esta práctica fue:

unsloth/Llama-3.2-1B-Instruct

Este modelo pertenece a la familia LLaMA y está basado en la arquitectura Transformer. Al tratarse de una versión *Instruct*, está diseñado para seguir instrucciones y generar respuestas conversacionales, por lo que resulta adecuado para construir un chatbot especializado. En esta práctica, el modelo fue adaptado para responder problemas técnicos con una estructura específica de salida: diagnóstico y solución.

Dataset Utilizado

El conjunto de datos utilizado corresponde a un dataset de tickets de soporte técnico. Cada ejemplo contiene una instrucción general, una entrada con el problema reportado por el usuario y una salida esperada con el diagnóstico y la solución. El dataset fue almacenado en formato JSON y cargado desde Google Drive para su uso durante el entrenamiento.

La práctica se enfocó en el comportamiento asignado:

IT Helpdesk Chatbot

Dataset: IT support tickets

Objetivo: diagnosticar y sugerir soluciones a problemas técnicos.

Entorno de Ejecución

El desarrollo de la práctica se realizó principalmente en **Google Colab**, utilizando GPU para acelerar el entrenamiento del modelo. El modelo ajustado fue almacenado en **Google Drive**, permitiendo conservar el resultado del entrenamiento y reutilizarlo posteriormente sin necesidad de repetir todo el proceso.

Para la fase de prueba, se utilizó una interfaz web desarrollada con **Gradio** dentro de Colab. Esta interfaz permitió validar el funcionamiento del chatbot de manera visual e interactiva, mostrando respuestas automáticas a preguntas de prueba y permitiendo que el usuario escribiera nuevos problemas técnicos.

Desarrollo de la Practica

El desarrollo de esta práctica consistió en la construcción, adaptación y prueba de un chatbot especializado en soporte técnico. El objetivo principal fue utilizar un modelo de lenguaje preentrenado y ajustarlo mediante LoRA para que pudiera recibir problemas técnicos escritos por el usuario, diagnosticarlos y proponer soluciones claras.

La práctica se llevó a cabo en Google Colab, utilizando un dataset en formato JSON basado en tickets de soporte técnico. Posteriormente, el modelo ajustado fue almacenado en Google Drive y probado mediante una interfaz gráfica desarrollada con Gradio.

1. Construcción del Dataset

Para el entrenamiento del chatbot se utilizó un dataset personalizado basado en tickets de soporte técnico. Cada registro representa un caso de soporte en el que un usuario reporta un problema técnico y el modelo debe responder con un diagnóstico y una solución.

El dataset fue almacenado en formato JSON y estructurado mediante tres campos principales:

```
{
  "instruction": "Act as an IT Helpdesk support chatbot. Diagnose the user's technical problem and provide a clear, brief, and c",
  "input": "I cannot connect to the WiFi.",
  "output": "Diagnosis: possible wireless connection or authentication problem. Solution: verify that WiFi is enabled, confirm t",
},
{
  "instruction": "Act as an IT Helpdesk support chatbot. Diagnose the user's technical problem and provide a clear, brief, and c",
  "input": "My laptop does not connect to the wireless network.",
  "output": "Diagnosis: possible wireless connection or authentication problem. Solution: verify that WiFi is enabled, confirm t",
},
{
  "instruction": "Act as an IT Helpdesk support chatbot. Diagnose the user's technical problem and provide a clear, brief, and c",
  "input": "The WiFi asks for a password but it does not connect.",
  "output": "Diagnosis: possible wireless connection or authentication problem. Solution: verify that WiFi is enabled, confirm t",
},
{
  "instruction": "Act as an IT Helpdesk support chatbot. Diagnose the user's technical problem and provide a clear, brief, and c",
  "input": "I forgot the WiFi network and now I cannot reconnect.",
  "output": "Diagnosis: possible wireless connection or authentication problem. Solution: verify that WiFi is enabled, confirm t",
},
{
  "instruction": "Act as an IT Helpdesk support chatbot. Diagnose the user's technical problem and provide a clear, brief, and c",
  "input": "My computer says it is unable to join the WiFi.",
  "output": "Diagnosis: possible wireless connection or authentication problem. Solution: verify that WiFi is enabled, confirm t",
},
{
  "instruction": "Act as an IT Helpdesk support chatbot. Diagnose the user's technical problem and provide a clear, brief, and c",
  "input": "The office WiFi is not working on my laptop.",
  "output": "Diagnosis: possible wireless connection or authentication problem. Solution: verify that WiFi is enabled, confirm t",
},
{
  "instruction": "Act as an IT Helpdesk support chatbot. Diagnose the user's technical problem and provide a clear, brief, and c",
  "input": "The school WiFi does not let me connect.",
  "output": "Diagnosis: possible wireless connection or authentication problem. Solution: verify that WiFi is enabled, confirm t",
},
{
  "instruction": "Act as an IT Helpdesk support chatbot. Diagnose the user's technical problem and provide a clear, brief, and c",
  "input": "My device keeps trying to connect to WiFi but fails.",
  "output": "Diagnosis: possible wireless connection or authentication problem. Solution: verify that WiFi is enabled, confirm t",
},
{
  "instruction": "Act as an IT Helpdesk support chatbot. Diagnose the user's technical problem and provide a clear, brief, and c",
  "input": "I get an authentication error when connecting to WiFi.",
  "output": "Diagnosis: possible wireless connection or authentication problem. Solution: verify that WiFi is enabled, confirm t",
},
}
```

Figura 1: Primeros datos del dataset

El campo instruction indica el comportamiento esperado del modelo, el campo input contiene el problema reportado por el usuario y el campo output contiene la respuesta esperada. Esta respuesta mantiene una estructura fija basada en dos partes: diagnóstico y solución.

El uso de este formato permite que el modelo aprenda no solo a identificar el tipo de problema, sino también a responder de manera ordenada y consistente. Los ejemplos del dataset incluyen problemas relacionados con conexión WiFi, lentitud del equipo, contraseñas, impresoras, correo electrónico, cámara, virus, errores de sistema, dispositivos USB y plataformas web.

2. Carga del Dataset

El dataset fue guardado en Google Drive para facilitar su acceso desde Google Colab. En el código de entrenamiento se define la ruta del archivo JSON mediante la variable `dataset_path`, ubicada en **`/content/drive/MyDrive/datasets/it_helpdesk_dataset_finetuning.json`**.

Antes de iniciar el entrenamiento, el programa verifica que el archivo exista en la ruta especificada. Si el archivo no se encuentra, se genera un error para evitar continuar con un entrenamiento incompleto. Posteriormente, el archivo se abre con la biblioteca json para revisar su contenido y mostrar el número total de ejemplos del dataset.

Después de esta validación, el dataset se carga utilizando la función `load_dataset` de la biblioteca Hugging Face Datasets. Esta función convierte el archivo JSON en un objeto compatible con el proceso de entrenamiento. Además, se empleó la opción

download_mode="force_redownload" para asegurar que Colab leyera la versión actual del archivo y no una versión almacenada en caché

```
folders_to_delete = [
    lora_save_path,
    merged_save_path,
    training_output_dir
]

for folder in folders_to_delete:
    if os.path.exists(folder):
        shutil.rmtree(folder)
        print("Carpeta eliminada:", folder)

print("Limpieza completada.")

if not os.path.exists(dataset_path):
    raise FileNotFoundError(
        "No se encontró el dataset en:\n"
        f"{dataset_path}"
    )

with open(dataset_path, "r", encoding="utf-8") as f:
    data = json.load(f)

print("Dataset encontrado:", dataset_path)
print("Total de ejemplos:", len(data))
print("\nPrimer ejemplo:")
print(json.dumps(data[0], ensure_ascii=False, indent=2))

raw_dataset = load_dataset(
    "json",
    data_files={"train": dataset_path},
    download_mode="force_redownload"
)

train_dataset = raw_dataset["train"]

print("\nEjemplos usados para entrenamiento:", len(train_dataset))
```

Figura 2: Carga y verificación de parámetros en el dataset

3. Carga del Modelo Base

El modelo base utilizado fue **unsloth/Llama-3.2-1B-Instruct**, una versión de LLaMA diseñada para seguir instrucciones y generar respuestas conversacionales. Este modelo fue seleccionado porque, al ser una versión *Instruct*, ya cuenta con una capacidad previa para interpretar instrucciones y responder como asistente.

En el código se define el identificador del modelo mediante la variable `base_model_id`. También se establecen las rutas donde se guardará el adaptador LoRA y el modelo fusionado al finalizar el entrenamiento.

El tokenizador se carga con `AutoTokenizer.from_pretrained`. En caso de que el tokenizador no cuente con un token de relleno definido, se asigna el token de fin de secuencia como `pad_token`. Esto es necesario para que el modelo pueda procesar lotes de ejemplos con longitudes diferentes durante el entrenamiento.

Posteriormente, el modelo base se carga mediante `AutoModelForCausalLM.from_pretrained`, utilizando precisión `float16` y `device_map="auto"`. Esto permite aprovechar los recursos disponibles en Colab, especialmente la GPU, para distribuir automáticamente el modelo en el dispositivo correspondiente.

```
tokenizer = AutoTokenizer.from_pretrained(base_model_id)

if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

tokenizer.padding_side = "right"

print("Tokenizer cargado correctamente.")

base_model = AutoModelForCausalLM.from_pretrained(
    base_model_id,
    torch_dtype=torch.float16,
    device_map="auto"
)

base_model.config.use_cache = False

print("Modelo base cargado sin bitsandbytes:", base_model_id)

print("Modelo base cargado:", base_model_id)
```

Figura 3: Carga del tokenizador y del modelo

4. Formato Conversacional del Entrenamiento

Para que el modelo aprendiera el comportamiento de un chatbot, cada ejemplo del dataset fue transformado a un formato conversacional. Este formato incluye tres partes: un mensaje de sistema, un mensaje del usuario y una respuesta del asistente.

El mensaje de sistema define el rol del modelo como chatbot de soporte técnico. También indica que siempre debe responder en inglés y que la respuesta debe incluir un diagnóstico y una solución con el formato específico: **Diagnosis: ... Solution: ...**

Este mensaje de sistema se encuentra definido en el código mediante la variable ***system_message***.

Para construir el formato de entrenamiento se implementó la función `build_chat_text`. Esta función toma cada ejemplo del dataset y construye dos versiones del texto: una versión completa, que incluye sistema, usuario y asistente, y una versión de prompt, que solo incluye sistema y usuario.

La versión completa se utiliza para que el modelo conozca cuál debe ser la respuesta esperada, mientras que la versión del prompt permite identificar qué parte del texto corresponde únicamente a la entrada. Esta separación es importante porque durante el entrenamiento no se busca que el modelo aprenda a copiar el mensaje del sistema ni la pregunta del usuario, sino que aprenda a generar la respuesta del asistente.

```
system_message = (
    "You are an IT Helpdesk support chatbot. "
    "Always answer in English. "
    "Your answer must include a diagnosis and a clear solution. "
    "Use this exact format: Diagnosis: ... Solution: ..."
)

def build_chat_text(example):
    user_message = example["input"]
    assistant_message = example["output"]

    messages_full = [
        {
            "role": "system",
            "content": system_message
        },
        {
            "role": "user",
            "content": user_message
        },
        {
            "role": "assistant",
            "content": assistant_message
        }
    ]

    messages_prompt = [
        {
            "role": "system",
            "content": system_message
        },
        {
            "role": "user",
            "content": user_message
        }
    ]
```

Figura 4: Construcción del Formato conversacional del dataset usado

5. Tokenización y Enmascaramiento

Una vez construido el formato conversacional, los textos fueron tokenizados. La tokenización consiste en convertir el texto en unidades numéricas llamadas tokens, que son las representaciones que el modelo puede procesar.

En esta práctica se utilizó una longitud máxima de 512 tokens, definida mediante la variable `max_length`. Para cada ejemplo se tokenizó tanto el texto completo como el prompt.

Además, se aplicó una estrategia de enmascaramiento mediante la función `tokenize_and_mask`. Esta función asigna el valor -100 a los tokens correspondientes al mensaje de sistema y al mensaje del usuario. En PyTorch, el valor -100 indica que esos tokens no deben tomarse en cuenta al calcular la pérdida durante el entrenamiento.

De esta manera, el modelo solo aprende a generar la respuesta del asistente. Esto evita que el modelo memorice o repita el prompt completo y permite que se enfoque en producir diagnósticos y soluciones.

```
print("\nEjemplo de entrenamiento:")
print(formatted_dataset[0]["full_text"])

max_length = 512

def tokenize_and_mask(example):
    full = tokenizer(
        example["full_text"],
        truncation=True,
        max_length=max_length,
        padding="max_length"
    )

    prompt = tokenizer(
        example["prompt_text"],
        truncation=True,
        max_length=max_length,
        padding=False
    )

    input_ids = full["input_ids"]
    attention_mask = full["attention_mask"]

    labels = input_ids.copy()

    prompt_len = len(prompt["input_ids"])

    labels[:prompt_len] = [-100] * prompt_len

    labels = [
        token if mask == 1 else -100
        for token, mask in zip(labels, attention_mask)
    ]

    return {
        "input_ids": input_ids,
        "attention_mask": attention_mask,
        "labels": labels
    }

tokenized_dataset = formatted_dataset.map(
    tokenize_and_mask,
    remove_columns=formatted_dataset.column_names
)

print("\nDataset tokenizado correctamente.")
```

Figura 5: Tokenizar el dataset

6. Configuración de LoRA

Para adaptar el modelo de manera eficiente se utilizó la técnica LoRA. Esta técnica permite entrenar únicamente un conjunto reducido de parámetros adicionales, sin modificar todos los pesos del modelo base. Esto reduce el consumo de memoria y permite ajustar modelos grandes con recursos más limitados.

En la práctica, LoRA fue configurado con los siguientes parámetros:

```
lora_config = LoraConfig(  
    r=8,  
    lora_alpha=16,  
    target_modules=[  
        "q_proj",  
        "k_proj",  
        "v_proj",  
        "o_proj"  
    ],  
    lora_dropout=0.05,  
    bias="none",  
    task_type="CAUSAL_LM"  
)  
  
model = get_peft_model(base_model, lora_config)  
  
print("\nParámetros entrenables:")  
model.print_trainable_parameters()
```

Figura 6: Configuración del LoRA

7. Configuración del Entrenamiento

Una vez tokenizado el dataset y aplicado el enmascaramiento correspondiente, se configuró el proceso de entrenamiento del modelo. Para ello, se utilizó la clase `TrainingArguments` de la biblioteca `Transformers`, la cual permite definir los principales parámetros del entrenamiento.

En esta práctica se estableció un tamaño de lote por dispositivo de 2, junto con una acumulación de gradiente de 4. Esta configuración permite simular un lote efectivo mayor sin requerir una cantidad excesiva de memoria en GPU. Además, se definieron 6 épocas de entrenamiento, con una

tasa de aprendizaje de $2e-4$, lo cual permitió realizar una adaptación suficiente del modelo al dominio de soporte técnico.

La configuración utilizada fue la siguiente:

```
training_args = TrainingArguments(  
    output_dir=training_output_dir,  
  
    per_device_train_batch_size=2,  
    gradient_accumulation_steps=4,  
  
    num_train_epochs=6,  
    learning_rate=2e-4,  
  
    logging_steps=10,  
    save_strategy="epoch",  
    save_total_limit=2,  
  
    report_to="none",  
  
    fp16=True,  
    optim="adamw_torch",  
  
    warmup_ratio=0.05,  
    lr_scheduler_type="cosine",  
  
    gradient_checkpointing=True  
)
```

Figura 7: Configuración de entrenamiento del modelo

El uso de `fp16=True` permitió trabajar con precisión de 16 bits, reduciendo el consumo de memoria durante el entrenamiento. Asimismo, se utilizó el optimizador `adamw_torch`, el cual es adecuado para el ajuste de modelos neuronales basados en Transformers. También se incluyó un calentamiento inicial mediante `warmup_ratio=0.05` y un planificador de tasa de aprendizaje tipo coseno, con el objetivo de estabilizar el proceso de aprendizaje.

Otro parámetro importante fue `gradient_checkpointing=True`, el cual ayuda a disminuir el uso de memoria durante el entrenamiento al recalcul ciertos valores intermedios en lugar de almacenarlos todos. Esta configuración fue útil debido a que se trabajó con un modelo

de lenguaje de tamaño considerable dentro de un entorno de Colab. Estos parámetros aparecen definidos en el código de entrenamiento mediante la clase `TrainingArguments`.

8. Entrenamiento del Modelo

Después de definir los argumentos de entrenamiento, se creó un objeto `Trainer`, encargado de coordinar el proceso de ajuste del modelo. Este objeto recibió como parámetros el modelo con LoRA aplicado, la configuración de entrenamiento y el dataset tokenizado.

```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=tokenized_dataset  
)  
  
print("\nIniciando entrenamiento LoRA con Llama...")  
  
trainer.train()  
  
print("\nEntrenamiento finalizado.")
```

Figura 8. Entrenamiento del modelo

Durante esta etapa, el modelo no modificó todos sus parámetros originales, sino únicamente los adaptadores LoRA agregados a las capas de atención. Esto permitió adaptar el comportamiento del modelo a la tarea de soporte técnico sin necesidad de realizar un entrenamiento completo del modelo base.

El objetivo del entrenamiento fue que el modelo aprendiera a generar respuestas estructuradas a partir de tickets de soporte técnico, manteniendo el formato:

Diagnosis: ... Solution: ...

De esta forma, el chatbot no solamente responde de manera conversacional, sino que organiza la salida en dos partes: primero identifica el posible origen del problema y después propone una solución inicial. En el código, el entrenamiento se ejecuta después de imprimir

el mensaje “Iniciando entrenamiento LoRA con Llama...”, y finaliza cuando el proceso concluye correctamente.

9. Almacenamiento del Adaptador LoRA

Una vez finalizado el entrenamiento, el adaptador LoRA fue guardado en Google Drive para conservar los parámetros aprendidos. Esto evita tener que repetir el entrenamiento cada vez que se desea utilizar el chatbot.

El adaptador se guardó mediante las siguientes instrucciones:

```
model.save_pretrained(lora_save_path)
tokenizer.save_pretrained(lora_save_path)
```

Figura 9: Modelo LoRA guardado

Guardar el adaptador por separado resulta útil porque permite cargarlo posteriormente sobre el modelo base. Sin embargo, para facilitar las pruebas, también se realizó un

proceso de fusión entre el adaptador LoRA y el modelo base. El código guarda el adaptador entrenado y el tokenizador dentro de la ruta definida por `lora_save_path`.

10. Fusión del Adaptador con el Modelo Base

Después de guardar el adaptador LoRA, se realizó la fusión con el modelo base. Este proceso permite obtener un modelo final que ya contiene los ajustes aprendidos, sin necesidad de cargar por separado el modelo base y el adaptador.

Antes de realizar la fusión, se liberó memoria eliminando las variables del modelo anterior y limpiando la memoria de CUDA:

```
del model
del base_model
torch.cuda.empty_cache()

base_model_for_merge = AutoModelForCausalLM.from_pretrained(
    base_model_id,
    torch_dtype=torch.float16,
    device_map="auto"
)

merged_model = PeftModel.from_pretrained(
    base_model_for_merge,
    lora_save_path
)

merged_model = merged_model.merge_and_unload()

merged_model.save_pretrained(
    merged_save_path,
    safe_serialization=True
)

tokenizer.save_pretrained(merged_save_path)

print("\nModelo fusionado guardado en:")
print(merged_save_path)
```

Figura 10. Merge del adaptador y el modelo

Posteriormente, se cargó nuevamente el modelo base y se aplicó el adaptador LoRA entrenado mediante `PeftModel.from_pretrained`. Una vez cargado el adaptador, se utilizó el método `merge_and_unload()` para fusionar los pesos LoRA con el modelo base.

Finalmente, el modelo fusionado fue guardado en Google Drive con la opción `safe_serialization=True`, lo que permite almacenarlo en un formato seguro y reutilizable.

La ruta final del modelo fusionado fue:

`/content/drive/MyDrive/it_helpdesk_llama_lora_merged_model`

Esta carpeta es la que posteriormente se utilizó para cargar el chatbot en la interfaz de prueba con Gradio. En el código se observa que el modelo base se vuelve a cargar, se aplica el adaptador LoRA y después se guarda el modelo fusionado en la ruta `merged_save_path`.

11. Ventajas del Modelo Fusionado

El almacenamiento del modelo fusionado permitió simplificar la etapa de prueba, ya que no fue necesario volver a cargar manualmente el adaptador LoRA sobre el modelo base. Esto facilitó la creación de la interfaz interactiva, debido a que bastó con cargar el modelo final desde Google Drive.

Además, al guardar el modelo en Google Drive, el resultado del entrenamiento permanece disponible aunque se cierre la sesión de Colab. Esto es importante porque el entrenamiento puede consumir tiempo y recursos computacionales, por lo que conservar el modelo entrenado permite reutilizarlo posteriormente sin repetir todo el proceso.

En resumen, esta etapa permitió pasar de un modelo base preentrenado a un modelo especializado en el dominio de soporte técnico, capaz de recibir reportes de usuarios y generar respuestas con diagnóstico y solución.

Resultados

Una vez finalizado el entrenamiento del modelo y guardado el modelo fusionado en Google Drive, se realizaron pruebas para evaluar el comportamiento del chatbot de soporte técnico. El objetivo de estas pruebas fue comprobar si el modelo era capaz de recibir problemas técnicos escritos por el usuario y generar respuestas estructuradas con un diagnóstico y una solución.

El modelo fue probado mediante una interfaz gráfica desarrollada con Gradio en Google Colab. Esta interfaz permitió visualizar la conversación entre el usuario y el chatbot,

ejecutar preguntas automáticas de prueba y escribir nuevas entradas de manera interactiva. Para la prueba se cargó el modelo fusionado desde la ruta:

`/content/drive/MyDrive/it_helpdesk_llama_lora_merged_model`

En el código de prueba, el modelo se carga junto con su tokenizador utilizando `AutoTokenizer` y `AutoModelForCausalLM`, empleando `torch_dtype=torch.float16` y `device_map="auto"` para aprovechar la GPU disponible en Colab.

La interfaz fue configurada para ejecutar un conjunto de preguntas automáticas relacionadas con problemas comunes de soporte técnico. Estas preguntas incluyen fallas de conexión WiFi, lentitud del equipo, recuperación de contraseña, problemas de impresora, errores de software, fallas de correo electrónico, cámara, malware, errores de

Windows, acceso a Moodle, errores de servidor, dispositivos USB, micrófono y enlaces sospechosos en correos.

A continuación, se muestran algunos ejemplos representativos de las pruebas realizadas:

Entrada del usuario	Respuesta esperada del chatbot
I cannot connect to the WiFi	El chatbot debe diagnosticar un posible problema de conexión inalámbrica o autenticación, y sugerir revisar que el WiFi esté activado, verificar la red, escribir nuevamente la contraseña, olvidar la red y reconectarse.
My computer is very slow	El chatbot debe diagnosticar un posible problema de rendimiento y sugerir cerrar programas innecesarios, reiniciar el equipo, revisar almacenamiento, actualizaciones y ejecutar un antivirus.
I forgot my password	El chatbot debe diagnosticar un problema de autenticación y sugerir verificar usuario, revisar Bloq Mayús, usar recuperación de contraseña o contactar al administrador.
The printer does not print	El chatbot debe diagnosticar un problema de impresora, conexión, controlador o cola de impresión, y sugerir revisar papel, tinta, conexión, trabajos pendientes y reiniciar la impresora.
I opened a suspicious link in an email	El chatbot debe diagnosticar un posible incidente de phishing o malware y sugerir no abrir más enlaces, cambiar la contraseña, ejecutar antivirus y reportar el incidente a soporte.

Durante la ejecución de la interfaz, las respuestas generadas por el modelo mantuvieron el formato solicitado en el mensaje de sistema:

Diagnosis: ... Solution: ...

Este formato permitió que las respuestas fueran claras y fáciles de interpretar, separando la identificación del problema de las acciones sugeridas para resolverlo. El comportamiento del modelo fue definido mediante un `system_message`, en el cual se especifica que debe actuar como chatbot de soporte técnico, responder en inglés y generar una respuesta con diagnóstico y solución.

Además de las pruebas automáticas, la interfaz permitió ingresar problemas técnicos propios mediante un cuadro de texto. Al enviar una pregunta, la función `respond` agrega el

mensaje del usuario al historial, llama a la función ask_chatbot y muestra la respuesta generada por el modelo en la conversación.

La interfaz final incluyó botones para enviar mensajes, limpiar la conversación, volver a ejecutar las pruebas automáticas y probar ejemplos específicos como WiFi, contraseña, impresora y phishing. Esto permitió validar el funcionamiento del chatbot de manera más sencilla y visual.

En general, los resultados muestran que el modelo fue capaz de generar respuestas coherentes para diferentes tipos de problemas técnicos. Las respuestas no se limitaron a repetir la entrada del usuario, sino que incluyeron una interpretación del problema y una propuesta de solución. Esto indica que el ajuste mediante LoRA permitió adaptar el modelo base al dominio específico de soporte técnico.

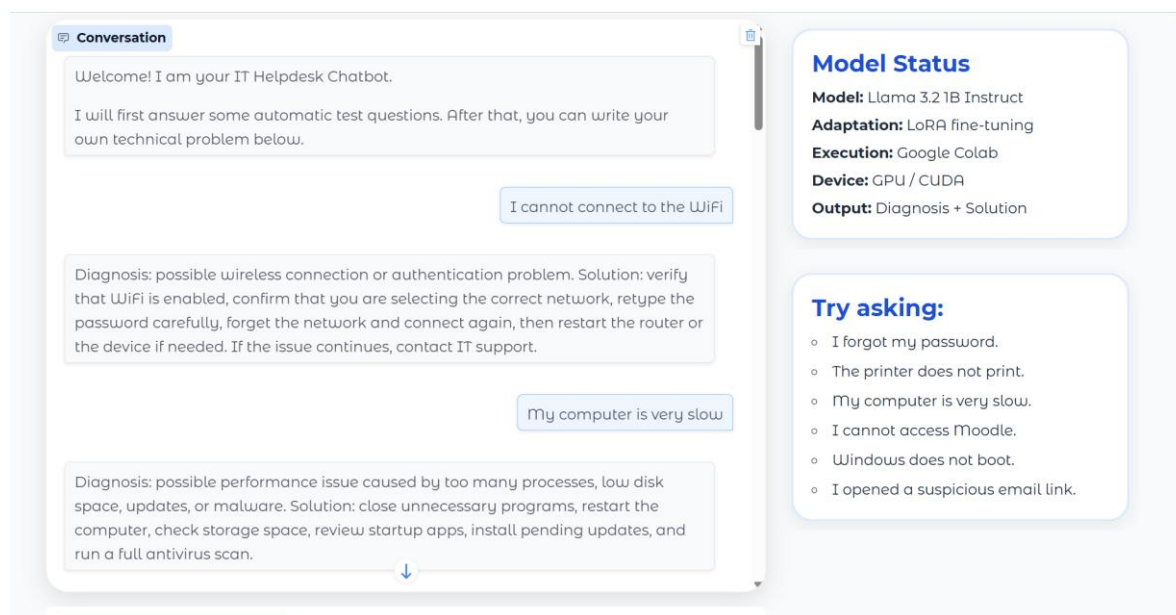


Figura 11. Respuestas del modelo

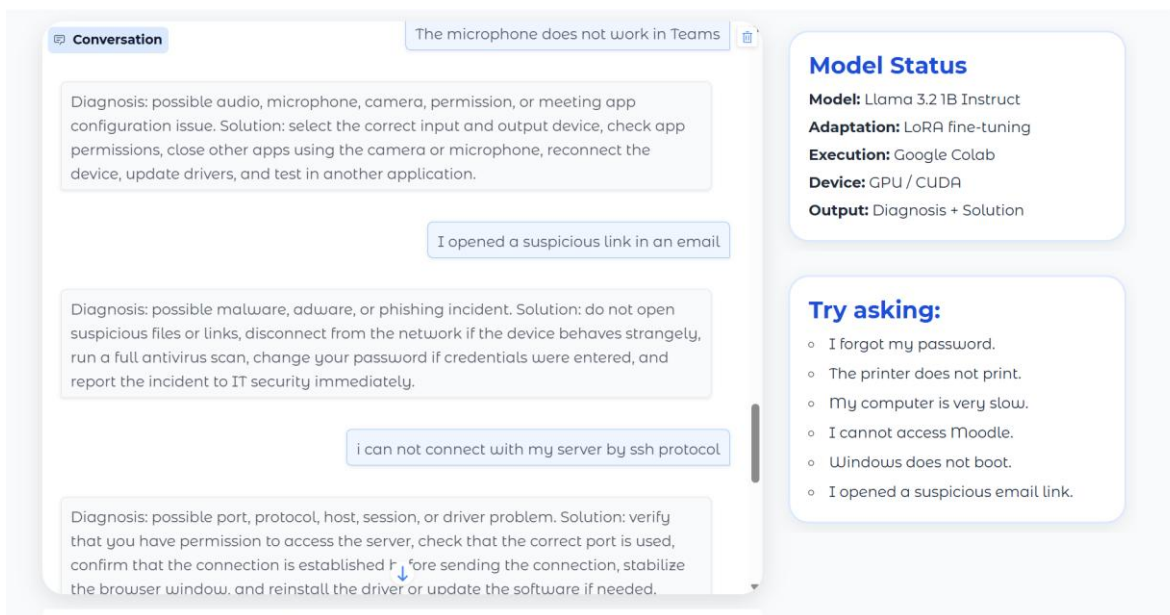


Figura 12. Respuestas del modelo

Análisis de Resultados

Los resultados obtenidos muestran que el chatbot fue capaz de responder de manera adecuada a distintos problemas técnicos reportados por el usuario. A diferencia de un sistema basado únicamente en reglas, el modelo ajustado mediante LoRA logró generar respuestas en lenguaje natural, manteniendo una estructura clara y consistente.

Uno de los principales aspectos positivos observados fue que el modelo mantuvo el formato solicitado desde el mensaje de sistema:

Diagnosis: ... Solution: ...

Este formato permitió separar la identificación del problema de la recomendación sugerida. De esta manera, cada respuesta del chatbot resulta más fácil de interpretar para el usuario, ya que primero se presenta una posible causa del problema y posteriormente se indican pasos concretos para solucionarlo. Esta estructura fue definida tanto en el entrenamiento como en la interfaz de prueba mediante el `system_message`, donde se indica que el chatbot debe actuar como soporte técnico y responder con diagnóstico y solución.

Durante las pruebas, el modelo respondió correctamente a problemas comunes como fallas de conexión WiFi, lentitud del equipo, recuperación de contraseña, errores de impresora, problemas de correo electrónico, cámara en Zoom, posibles infecciones por malware, errores de arranque de Windows y fallas en plataformas web. Esto indica que el dataset utilizado permitió cubrir varias categorías representativas del área de IT Helpdesk.

La interfaz de prueba permitió observar de forma clara el comportamiento del modelo ante diferentes entradas. Las preguntas automáticas incluidas en la lista `test_questions` funcionaron como un conjunto de validación cualitativa, ya que abarcaron distintos tipos de incidentes técnicos.

También se observó que el modelo no se limitó a copiar la entrada del usuario, sino que generó una respuesta extendida y contextualizada. Por ejemplo, ante un problema de WiFi, el chatbot no solo identifica una posible falla de conexión, sino que también sugiere revisar si el WiFi está activado, verificar la red, escribir correctamente la contraseña, olvidar la red y reconectarse. Este comportamiento demuestra que el modelo aprendió a relacionar los reportes de soporte con soluciones prácticas.

Otro punto importante fue el uso de generación determinista mediante el parámetro:

`do_sample=False`

Esta configuración permite que el modelo produzca respuestas más consistentes para una misma entrada, lo cual es conveniente en un sistema de soporte técnico. En este tipo de aplicaciones, se busca que las respuestas sean estables, ordenadas y confiables, evitando variaciones innecesarias o respuestas demasiado creativas. En el código de la interfaz, este parámetro se utiliza dentro de la función `ask_chatbot`.

El uso de LoRA resultó adecuado para esta práctica porque permitió adaptar un modelo preentrenado sin modificar completamente todos sus parámetros. Esto redujo el costo computacional del entrenamiento y facilitó el uso de un modelo basado en LLaMA dentro de Google Colab. La configuración aplicada a los módulos `q_proj`, `k_proj`, `v_proj` y `o_proj` permitió ajustar partes importantes del mecanismo de atención del modelo, manteniendo una adaptación ligera y eficiente.

Además, el proceso de enmascaramiento durante la tokenización fue relevante para evitar que el modelo aprendiera a repetir el mensaje del sistema o el problema del usuario. Al asignar -100 a los tokens correspondientes al prompt, el entrenamiento se enfocó únicamente en la respuesta del asistente. Esto ayudó a que el modelo aprendiera a generar la salida esperada en lugar de memorizar toda la conversación de entrada.

La interfaz desarrollada con Gradio permitió validar el modelo de forma visual e interactiva. Gracias a esta interfaz, fue posible ejecutar pruebas automáticas, escribir problemas personalizados y observar las respuestas generadas por el chatbot en un formato conversacional. Esto facilitó la evaluación cualitativa del sistema, ya que no fue necesario interactuar únicamente desde la consola de Colab.

Sin embargo, también se identificaron algunas limitaciones. Al tratarse de un dataset construido para fines prácticos, el modelo puede presentar respuestas menos precisas ante problemas técnicos muy específicos o que no estén representados dentro de los

ejemplos de entrenamiento. Por esta razón, el desempeño del chatbot depende directamente de la variedad y calidad del dataset.

Otra limitación observada fue la dificultad para ejecutar el modelo en un entorno local sin GPU. Aunque el modelo fusionado podía cargarse desde Google Drive y funcionar adecuadamente en Colab, la ejecución local resultó más demandante en memoria y procesamiento. Esto muestra que, aunque LoRA reduce el costo del entrenamiento, la inferencia de modelos de lenguaje todavía puede requerir recursos considerables si se trabaja con modelos en formato Hugging Face.

En general, los resultados fueron satisfactorios para el objetivo de la práctica. El chatbot logró cumplir con la tarea asignada: recibir tickets o problemas de soporte técnico, diagnosticar la posible causa y sugerir soluciones claras. Además, el uso de una interfaz gráfica permitió demostrar el funcionamiento del sistema de manera más visual y cercana a una aplicación real de IT Helpdesk.

Conclusiones y recomendaciones

En esta práctica se logró desarrollar un chatbot especializado en soporte técnico, cuyo objetivo principal fue diagnosticar problemas técnicos reportados por usuarios y sugerir soluciones claras. Para ello, se utilizó un modelo de lenguaje preentrenado de la familia LLaMA, específicamente unsloth/Llama-3.2-1B-Instruct, el cual fue adaptado mediante la técnica LoRA para responder dentro del dominio de IT Helpdesk.

El uso de un modelo tipo *Instruct* permitió partir de una base previamente entrenada para seguir instrucciones y generar respuestas conversacionales. Esto resultó importante, ya que el chatbot no debía limitarse a clasificar un problema, sino generar una respuesta completa que incluyera una interpretación del incidente y una recomendación de solución.

La técnica LoRA permitió realizar el ajuste fino del modelo de manera eficiente, modificando únicamente una pequeña parte de los parámetros del modelo base. Esto redujo el costo computacional del entrenamiento y facilitó el uso de un modelo de lenguaje dentro de Google Colab. En la configuración utilizada, LoRA fue aplicado sobre los módulos `q_proj`, `k_proj`, `v_proj` y `o_proj`, los cuales forman parte del mecanismo de atención del modelo Transformer.

El dataset utilizado, basado en tickets de soporte técnico, permitió entrenar al modelo con ejemplos representativos de problemas comunes, tales como fallas de WiFi, lentitud del equipo, contraseñas, impresoras, correo electrónico, cámara, malware, errores de sistema y problemas con plataformas web. Gracias a esto, el chatbot logró generar respuestas coherentes en diferentes escenarios de soporte técnico.

Una parte fundamental del desarrollo fue el formato conversacional utilizado durante el entrenamiento. Cada ejemplo fue estructurado con un mensaje de sistema, un mensaje del usuario y una respuesta del asistente. Además, se aplicó una estrategia de enmascaramiento para que el modelo aprendiera únicamente la respuesta esperada, evitando que copiara el prompt o la entrada del usuario.

Los resultados obtenidos muestran que el modelo fue capaz de responder siguiendo el formato solicitado:

Diagnosis: ... Solution: ...

Esta estructura permitió que las respuestas fueran ordenadas, interpretables y adecuadas para un contexto de soporte técnico. La separación entre diagnóstico y solución facilita que el usuario comprenda primero la posible causa del problema y después las acciones recomendadas para resolverlo.

La interfaz desarrollada con Gradio permitió probar el chatbot de manera visual e interactiva. Esta interfaz incluyó pruebas automáticas, botones de ejemplo y un cuadro de texto para ingresar problemas personalizados. De esta manera, fue posible validar el funcionamiento del modelo sin depender únicamente de una consola de código.

Como recomendación principal, se sugiere ampliar el dataset con una mayor cantidad de ejemplos y categorías. Aunque el modelo respondió correctamente a varios problemas comunes, su desempeño puede mejorar si se agregan más variantes lingüísticas y casos específicos de soporte técnico. Por ejemplo, se podrían incluir más ejemplos relacionados con redes institucionales, errores de software especializado, problemas de permisos, fallas de hardware, plataformas académicas y ciberseguridad.

También se recomienda mejorar el proceso de evaluación del modelo mediante métricas cuantitativas. En esta práctica la evaluación fue principalmente cualitativa, observando si las respuestas generadas eran coherentes y útiles. Sin embargo, en trabajos posteriores podrían utilizarse métricas automáticas o una evaluación manual con criterios como precisión del diagnóstico, utilidad de la solución, claridad de la respuesta y consistencia del formato.

Otra recomendación importante es optimizar la ejecución local del modelo. Durante las pruebas se observó que ejecutar el modelo fuera de Colab puede requerir una cantidad considerable de memoria, especialmente si se utiliza el formato original de Hugging Face. Para un uso más práctico en equipos con recursos limitados, sería conveniente convertir el modelo a un formato más ligero, como GGUF, o desplegarlo mediante una API en la nube.

Finalmente, se recomienda mejorar la interfaz para permitir conversaciones de múltiples turnos, historial persistente y clasificación de prioridad del incidente. Esto acercaría el sistema a una solución real de IT Helpdesk, donde no solo se genera una respuesta inicial, sino que también se da seguimiento al problema reportado por el usuario.

En conclusión, la práctica permitió aplicar de manera integral conceptos de procesamiento de lenguaje natural, modelos Transformer, fine-tuning eficiente y desarrollo de interfaces interactivas. El sistema resultante demuestra que un modelo preentrenado puede ser adaptado con LoRA para resolver una tarea específica de soporte técnico, generando respuestas útiles, estructuradas y orientadas a la solución de problemas.

Bibliografía

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M. A., Lacroix, T., Rozière, B., et al. (2023). *LLaMA: Open and Efficient Foundation Language Models*. Meta AI.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, L., & Chen, W. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv preprint arXiv:2106.09685.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., et al. (2020). *Transformers: State-of-the-Art Natural Language Processing*. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations.
- Hugging Face. (2026). *Transformers Documentation*. Hugging Face.
- Hugging Face. (2026). *Datasets Documentation*. Hugging Face.
- Hugging Face. (2026). *PEFT: Parameter-Efficient Fine-Tuning Documentation*. Hugging Face.
- PyTorch Team. (2026). *PyTorch Documentation*. PyTorch.
- Google. (2026). *Google Colaboratory Documentation*. Google.
- Gradio. (2026). *Gradio Documentation*. Gradio.
- Unsloth AI. (2026). *Llama-3.2-1B-Instruct Model Card*. Unsloth.
- Meta AI. (2024). *Llama 3.2 Model Documentation*. Meta AI.
- OpenAI. (2023). *ChatGPT and Conversational AI Systems*. OpenAI.
- Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing*. Stanford University.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.